

Privacy-preserving behavioral intelligence for distributed cloud systems via personalized federated autoencoders

Venkata Saikumar Thalupuru¹, Shubham Kumar², Santhoshini Pranathi Singaraju³, and Vishal Gupta⁴

¹ Department of Computer Science, Birla Institute of Technology and Science, Pilani, India

`p20200504@pilani.bits-pilani.ac.in`

² Department of Computer Science, Birla Institute of Technology and Science, Pilani, India

`f20220056@pilani.bits-pilani.ac.in`

³ RGUKT RK Valley, Idupulapaya, India

`rr200228@rguktrkv.ac.in`

⁴ Department of Computer Science, Birla Institute of Technology and Science, Pilani, India

`vishalgupta@pilani.bits-pilani.ac.in`

Abstract. Cloud storage platforms support diverse multimedia and collaborative workloads across organizations, yet conventional methods neglect user behavior’s role in shaping access patterns. Privacy regulations prohibit centralized aggregation of interaction traces, while standard federated learning algorithms like FedAvg fail under statistical heterogeneity from varied user roles. This paper introduces FedPAE (Federated Personalized AutoEncoder), an unsupervised framework for behavior-aware user profiling in federated settings. FedPAE employs a shared global encoder for common patterns and private local decoders for individual adaptation, augmented by an Adaptive Fine-Tuning (AF) mechanism to mitigate encoder drift and preserve global semantics, without sharing any raw user data with the server. Evaluated on the CMU CERT benchmark and anonymized cloud storage logs, FedPAE surpasses FedAvg, FedProx, and FedPer in anomaly detection accuracy across all thresholds (e.g., F1 gains of 5–13% points over FedAvg across all precision thresholds), confirming that the approach holds across heterogeneous client populations.

Keywords: Federated learning · Personalization · Unsupervised anomaly detection · Non-IID data · Privacy-preserving modeling · Cloud storage

1 Introduction

Cloud storage systems today handle real-time multimedia collaboration across large organizations, where traffic patterns closely reflect each user’s daily role. Yet most systems still react to demand spikes only after they occur. Learning

what “normal” looks like for different user roles would let the system anticipate demand instead.

Privacy rules and corporate data policies block centralized user logs [16]. Federated learning (FL) enables decentralized model training by keeping data local to each client and exchanging model updates instead of raw interaction logs [15], but standard algorithms like FedAvg create one shared model for all clients. When users in different roles behave very differently, this shared model fits no one well—this results in what we term “profile confusion,” where the shared model converges to a compromise that fails to represent any specific user role accurately [21]. Split-architecture personalized FL methods like FedPer [1] and FedRep [3] help with this, but they have only been tested on supervised tasks [8]. Their application to unsupervised behavioral profiling, where no anomaly labels exist at training time, has not been studied.

We propose **FedPAE** (Federated Personalized AutoEncoder): a shared *Global Encoder* captures role-agnostic patterns, while a private *Local Decoder* fits user-specific behavior. Empirically, naive training causes the encoder to *drift* as local decoders pull it different ways, so we add **Adaptive Fine-Tuning (AF)** to stabilize it before each joint round.

Contributions.

1. We introduce *FedPAE*, a personalized federated autoencoder for privacy-preserving behavioral profiling in distributed cloud storage systems. The framework separates a shared global encoder from private local decoders, allowing role-specific behavioral modeling without sharing raw user data.
2. We propose a lightweight *Adaptive Fine-Tuning (AF)* mechanism that stabilizes the shared encoder during federated training and mitigates representation drift caused by heterogeneous client behaviors.
3. We evaluate FedPAE on role-partitioned cloud storage logs and the CMU CERT benchmark, demonstrating consistent improvements over FedAvg, FedProx, and FedPer while providing interpretable insights through SHAP-based feature analysis.

2 Related Work

Personalized FL methods address statistical heterogeneity through different mechanisms. FedProx [10] adds a proximal regularization term to keep client models close to the global model, but still maintains a single shared model with no client-specific components; this limits its ability to capture role-specific behavioral distributions. Split-architecture methods such as FedPer [1] and FedRep [3] separate shared and private layers, achieving strong results for supervised classification [18,8]. However, their application to *unsupervised* behavioral profiling—where no anomaly labels exist—remains unexplored.

Federated autoencoders have been applied to anomaly detection in industrial IoT and power systems [9,17,19] by sharing encoders and keeping decoders local. Yet these methods target uniform sensor data from homogeneous devices and train the shared encoder directly against all local decoders simultaneously,

which induces representation drift under the role-based heterogeneity of enterprise networks. Approaches unsuitable for our setting include FedBN [11] (requires batch normalization bottlenecks absent in our autoencoders), pFedMe [4] (full per-client inner optimization loop, prohibitively costly), and Per-FedAvg [5] (requires ground-truth labels). FedPAE addresses all three gaps: decoder-split personalization, drift stabilization via AF, and fully unsupervised training.

3 Proposed Framework: FedPAE

FedPAE handles two key issues: users in different roles produce very different behavioral data, and we cannot centralize it. We split the model so the global encoder stays shared, while each local decoder remains private on the client. This cuts privacy risk—the decoder holds role-specific behavior, so the server only sees encoder gradients. The AF proximal term keeps $\|\theta_k - \theta^t\|$ small, making gradient inversion [22] much harder in practice [6,20].

3.1 Architecture

FedPAE divides a standard autoencoder into two parts:

- **Global Encoder** (E_θ): Compresses session log $x \in \mathbb{R}^{32}$ to $z \in \mathbb{R}^{16}$ via MLP [64, 32], ReLU ($\approx 3,200$ params). Shared and aggregated via FedAvg.
- **Local Decoder** (D_{ϕ_k}): Reconstructs $\hat{x}$ from z via MLP [32, 64], ReLU ($\approx 3,200$ params). Never uploaded; fits role-specific behavior.

Training minimizes per-client reconstruction error:

$$\mathcal{L}(\theta, \phi_k; x) = \|x - D_{\phi_k}(E_\theta(x))\|_2^2. \quad (1)$$

During AF the decoder is frozen and the encoder is updated with an augmented objective that adds a proximal penalty to prevent it from deviating too far from the current global parameters:

$$\mathcal{L}_{\text{AF}}(\theta; \phi_k, \theta^t) = \mathcal{L}(\theta, \phi_k) + \frac{\mu}{2} \|\theta - \theta^t\|_2^2, \quad (2)$$

where μ controls the proximity constraint. The gradient update during AF is $\theta_k \leftarrow \theta_k - \eta_{\text{AF}} \nabla_\theta \mathcal{L}_{\text{AF}}$, applied for τ_{AF} steps before resuming joint training of (θ_k, ϕ_k) .

3.2 Training Protocol

Each round t : (1) server broadcasts θ^t ; (2) if $t \geq T_{\text{start}}$, each client runs **AF** (Eq. 2)—freeze decoder, take τ_{AF} gradient steps, then jointly train (θ, ϕ_k) for E epochs; (3) clients return only θ_k^{t+1} ; server aggregates $\theta^{t+1} = \sum_k \frac{n_k}{N} \theta_k^{t+1}$. Algorithm 1 shows the full protocol.

Algorithm 1 FedPAE: Federated Personalized AutoEncoder

Require: K clients; rounds T ; T_{start} ; epochs E ; AF steps τ_{AF} ; rates η, η_{AF} ; coeff μ .

Ensure: $\theta^T, \{\phi_k^T\}_{k=1}^K$.

```
1: procedure SERVER
2:   Init  $\theta^0$ 
3:   for  $t = 1, \dots, T$  do
4:     Broadcast  $\theta^{t-1}$ ; collect  $\theta_k^t \leftarrow \text{CLIENT}(k, \theta^{t-1}, t) \forall k \in S_t$ 
5:      $\theta^t \leftarrow \sum_k \frac{n_k}{N} \theta_k^t$ 
6:   end for
7: end procedure
8: procedure CLIENT( $k, \theta_g, t$ )
9:    $\theta_k \leftarrow \theta_g$ 
10:  if  $t \geq T_{\text{start}}$  then
11:    Freeze  $\phi_k$ ;  $\tau_{AF}$  steps:  $\theta_k \leftarrow \eta_{AF} \nabla_{\theta} (\mathcal{L}_{\text{rec}} + \frac{\mu}{2} \|\theta_k - \theta_g\|^2)$ 
12:  end if
13:  Unfreeze; train  $(\theta_k, \phi_k)$ ,  $E$  epochs on  $\mathcal{D}_k$ 
14:  return  $\theta_k$ 
15: end procedure
```

3.3 Computational Overhead of AF

AF introduces τ_{AF} additional gradient steps per round, but these operate *only on the encoder* ($\approx 3,200$ parameters) with the decoder frozen, so no decoder back-propagation is computed. In our configuration ($\tau_{AF} = 3$, $E = 7$ joint epochs), the encoder-only AF passes account for roughly $3/(3+7) \approx 30\%$ of per-round gradient work, and each AF step is cheaper than a joint step because only half the model is active. Communication cost is identical to FedAvg and FedPer: only encoder weights ($\approx 3,200$ floats, ≈ 12.5 KB at fp32) are uploaded per round; the private decoder is never transmitted. On a commodity CPU client (Intel Core i5), the AF phase adds approximately 0.4s per round versus 1.1s for the joint training phase, making the overhead acceptable for periodic cloud storage audit cycles.

4 Experimental Setup

4.1 Datasets

Cloud Storage: Activity logs (uploads, downloads, edits, sharing) were anonymized by removing all user identifiers. Sessions are partitioned by role ($R \in \{\text{Intern, Analyst, Engineer, Finance, Director}\}$), giving natural non-IID distributions. Engineers generate high-volume uploads; Directors show broad, high-variability access. A 70/30 temporal train/test split is used with **StandardScaler** fitted *locally* per client.

CMU CERT: The CMU CERT Insider Threat Dataset [12] provides simulated organizational logs (logins, file access, email, device use) across a range of user

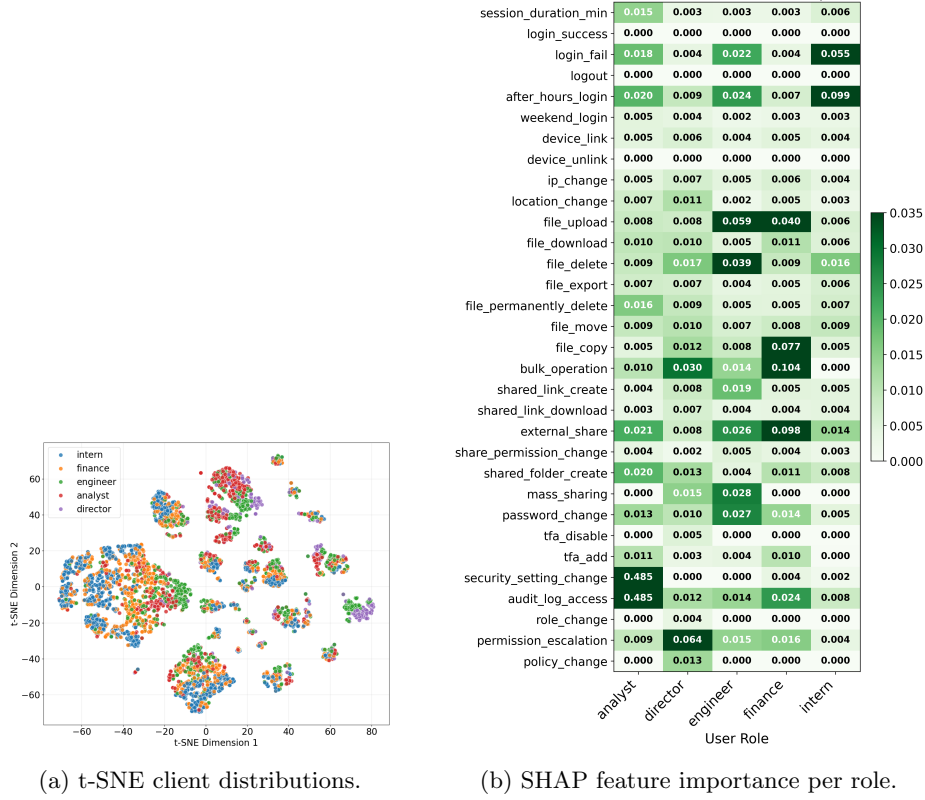


Fig. 1: Left: role-based non-IID client feature distributions (t-SNE, perplexity 30) [14]. Right: SHAP sensitivity heatmap—darker cells indicate stronger role-defining features.

roles. We use it for its realistic multi-role behavioral distributions, not its native insider threat labels. The same role-aware synthetic injection procedure is applied to create a labeled test set, identical to the cloud storage evaluation.

The 32-dimensional input spans five categories (Table 1). Fig. 1 (left) visualizes client feature distributions using t-SNE [14], confirming clear role-based clustering that motivates personalized decoding.

4.2 Role-Aware Anomaly Injection

Neither dataset provides anomaly labels suitable for role-based behavioral evaluation. Since real enterprise security datasets rarely release labeled anomalous activities due to privacy and confidentiality constraints, we construct a labeled test set by injecting role-aware anomalous sessions at rate $\rho = 0.2$. Table 2 lists the nine patterns; for example, Finance clients receive *Background Sync* (frequent exports) and Engineers receive *Project Consolidation* (bulk moves and

Table 1: Feature categories ($d = 32$).

Category	Features
Session Context	session_duration_min, after_hours_login, weekend_login, ip_change, location_change
Session Overhead	login_success/fail, logout, device_link/unlink
File Operations	upload, download, delete, move, copy, export, bulk_operation
Collaboration	external_share, mass_sharing, shared_link_create/download, shared_folder_create
Policy & Access	permission_escalation, role_change, policy_change, audit_log_access, tfa_disable/add

Table 2: Role-specific injected anomaly patterns.

Pattern	Primary Features
Metadata Indexing	audit_log_access, security_setting_change
Background Sync	file_export, external_share
Multi-Device Access	login_fail, after_hours_login, ip_change
Project Consolidation	file_upload, file_delete, file_move
Config Override	tfa_disable, policy_change
Bulk Retrieval	file_download, file_copy, bulk_operation
Off-Peak Workflow	after_hours_login, file_export
Priority Elevation	permission_escalation, role_change
Storage Cleanup	file_delete, file_permanently_delete

uploads). The same procedure is applied identically to both datasets, so any performance difference reflects only the difference in their underlying behavioral distributions.

4.3 Training Configuration and Baselines

Hyperparameters. Adam [7], $\eta = 0.005$, decay $\gamma = 0.90$; $T = 50$ rounds, $E = 7$ local epochs, $B = 64$; AF: $T_{\text{start}} = 10$, $\tau_{AF} = 3$, $\eta_{AF} = 0.001$, $\mu = 0.001$. All results are mean $\pm$ std over **5 independent runs**.

Baselines. *Local*: isolated per-client autoencoder (lower bound). *FedAvg* [15]: single global model. *FedProx* [10]: FedAvg with proximal term $\mu = 0.001$. *FedPer* [1]: same split architecture as FedPAE but without AF (primary ablation).

Metrics. Recall and F1 at Fixed Precision (R@P, F1@P) measure detection sensitivity across operating thresholds.

5 Results and Analysis

5.1 Burst Identification Performance

Tables 3 and 4 report Recall and F1 on the cloud storage dataset. FedPAE ranks first at every precision threshold. At $P = 0.85$: FedPAE Recall 0.680 vs. FedPer 0.643 ($\approx 4\%$ higher), FedProx 0.647, Local 0.583, FedAvg 0.497. The gap over

Table 3: Recall at Fixed Precision — Cloud Storage (mean $\pm$ std, 5 runs). Best per column in bold.

Method	P=0.65	P=0.70	P=0.75	P=0.80	P=0.85
Local	0.787 $\pm$.019	0.733 $\pm$.021	0.693 $\pm$.020	0.633 $\pm$.023	0.583 $\pm$.018
FedAvg	0.737 $\pm$.033	0.710 $\pm$.030	0.680 $\pm$.028	0.560 $\pm$.035	0.497 $\pm$.031
FedProx	0.790 $\pm$.015	0.757 $\pm$.014	0.717 $\pm$.013	0.693 $\pm$.014	0.647 $\pm$.014
FedPer	0.840 $\pm$.017	0.810 $\pm$.016	0.760 $\pm$.015	0.707 $\pm$.016	0.643 $\pm$.016
FedPAE	0.877$\pm$.012	0.830$\pm$.011	0.773$\pm$.012	0.730$\pm$.011	0.680$\pm$.012

Table 4: F1-Score at Fixed Precision — Cloud Storage (mean $\pm$ std, 5 runs). Best per column in bold.

Method	P=0.65	P=0.70	P=0.75	P=0.80	P=0.85
Local	0.712 $\pm$.018	0.717 $\pm$.016	0.721 $\pm$.015	0.708 $\pm$.017	0.693 $\pm$.018
FedAvg	0.691 $\pm$.028	0.705 $\pm$.026	0.713 $\pm$.024	0.659 $\pm$.030	0.627 $\pm$.028
FedProx	0.714 $\pm$.013	0.728 $\pm$.012	0.734 $\pm$.012	0.743 $\pm$.012	0.735 $\pm$.013
FedPer	0.734 $\pm$.015	0.751 $\pm$.014	0.755 $\pm$.014	0.750 $\pm$.015	0.732 $\pm$.015
FedPAE	0.747$\pm$.010	0.760$\pm$.010	0.762$\pm$.010	0.764$\pm$.010	0.756$\pm$.011

FedPer is the key result—both use the same split architecture, so the difference is entirely due to AF. Without it, the encoder is pulled in conflicting directions by each client’s decoder and loses the shared representation built up during warmup. FedAvg’s recall of 0.497—missing more than half of all anomalous bursts—is the direct consequence of profile confusion.

Fig. 2 shows convergence over 50 rounds. FedAvg collapses repeatedly, making it unsuitable for production. FedPAE reaches near-optimal performance within 10–15 rounds.

FedPAE vs. FedProx: decoder-split vs. regularization. FedProx improves over FedAvg by penalizing client models for deviating from the global model, but it does not alter the fundamental architecture: every client still optimizes the same shared model, meaning all users share both encoder *and* decoder weights. At $P = 0.85$, FedProx achieves Recall 0.647 (Table 3), a notable improvement over FedAvg (0.497), but it still falls short of FedPAE (0.680) by roughly 5%. The reason is structural: a proximal penalty can reduce weight divergence during training, but it cannot give different user roles their own reconstruction subspace. An Engineer and a Finance user optimizing the same decoder will always pull it toward a compromise that fits neither role precisely. FedPAE’s private local decoder removes this constraint, and the AF proximal term then serves a focused purpose—stabilizing only the shared encoder—rather than constraining the whole model. This architectural difference, not hyperparameter tuning, explains FedPAE’s consistent margin over FedProx across all five precision thresholds.

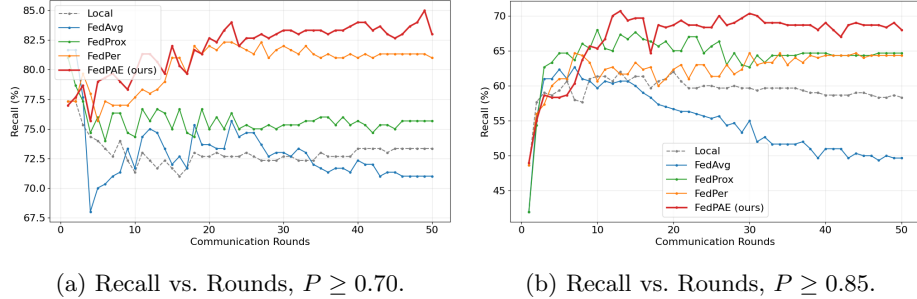


Fig. 2: Convergence over communication rounds. FedPAE (red) stabilizes within 10–15 rounds; FedAvg (blue) collapses repeatedly.

Table 5: CMU CERT Benchmark at $P = 0.85$ (mean $\pm$ std, 5 runs).

Method	Recall@P=0.85	F1@P=0.85
Local	0.512 $\pm$.022	0.614 $\pm$.019
FedAvg	0.441 $\pm$.034	0.574 $\pm$.027
FedProx	0.573 $\pm$.016	0.657 $\pm$.014
FedPer	0.581 $\pm$.017	0.661 $\pm$.015
FedPAE	0.624$\pm$.013	0.694$\pm$.012

5.2 Cross-Dataset Generalizability (CMU CERT)

Table 5 reports results on CMU CERT at $P = 0.85$. The absolute recall numbers are lower because CMU CERT has sparser behavioral patterns, but the ordering of methods is identical: FedPAE first (Recall 0.624 $\pm$ 0.013, F1 0.694 $\pm$ 0.012), with the same $\approx 4\%$ gap over FedPer as on the primary dataset. FedAvg again has the widest spread (± 0.034). The same pattern on a completely different dataset confirms that AF is solving a real problem, not fitting to quirks in the cloud storage logs.

5.3 Interpretability

Fig. 1 (right) shows SHAP feature importance per role [13]. The highest-weight features match Table 2: Engineers are flagged on `file_upload` and `file_delete`; Directors on `permission_escalation` and `policy_change`. When the model raises an alert, it is reacting to the specific behavioral shift we injected—not to noise. An operator can see exactly which activity triggered it.

6 Limitations and Future Work

Temporal Granularity. FedPAE uses session-aggregated vectors, missing within-session interaction sequences. Replacing the MLP encoder with LSTMs or Transformers could capture finer temporal patterns.

Privacy Guarantees. Formal differential privacy is not yet employed. Future work will add gradient clipping and noise injection to satisfy ϵ -DP [6,20].

Byzantine Robustness. The model assumes cooperative clients. Integrating Krum [2] or median-based aggregation would defend against model poisoning.

Scale. Current experiments involve five role-based client groups. Scaling to 50–100 heterogeneous clients is the natural next step, particularly to measure whether the AF overhead remains proportionally small as federation size increases.

7 Conclusion

This paper presented FedPAE, a federated autoencoder that keeps each user’s decoder private while sharing only the encoder. The Adaptive Fine-Tuning step prevents the shared encoder from drifting when trained jointly against decoders from different user roles — the FedPer baseline confirms this drift is real and costly. Across two datasets and five metrics, FedPAE detects more anomalous bursts at the same precision and wastes fewer resources on false alarms. The SHAP analysis confirms the model reacts to the right features for each role, so alerts are actionable. When users behave very differently from each other, one shared model is not enough. Giving each client its own decoder is a small architectural change, but it produces consistent and measurable improvements across heterogeneous behavioral patterns in distributed cloud systems.

Acknowledgements

The authors thank the Department of Computer Science, BITS Pilani, for providing computational infrastructure and support for this research.

References

1. Arivazhagan, M., Aggarwal, V., Singh, A., Choudhary, S.: Federated learning with personalization layers. arXiv preprint arXiv:1912.00818 (2019)
2. Blanchard, P., El Mhamdi, E., Guerraoui, R., Stainer, J.: Machine learning with adversaries: Byzantine tolerant gradient descent. In: Advances in Neural Information Processing Systems. vol. 30 (2017)
3. Collins, L., Hassani, H., Mokhtari, A., Shakkottai, S.: Exploiting shared representations for personalized federated learning. In: International Conference on Machine Learning. pp. 2089–2099 (2021)
4. Dinh, C., Tran, N., Nguyen, T.: Personalized federated learning with Moreau envelopes. Advances in Neural Information Processing Systems **33**, 21394–21405 (2020)
5. Fallah, A., Mokhtari, A., Ozdaglar, A.: Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach. Advances in Neural Information Processing Systems **33**, 3557–3568 (2020)
6. Geyer, R., Klein, T., Nabi, M.: Differentially private federated learning: A client level perspective. arXiv preprint arXiv:1712.07557 (2017)

7. Kingma, D., Ba, J.: Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980 (2014)
8. Kulkarni, V., Kulkarni, M., Pant, A.: Survey of personalization techniques for federated learning. In: World Conference on Smart Trends in Systems, Security and Sustainability. pp. 794–797 (2020)
9. Laridi, S., Palmer, G., Tam, K.: Enhanced federated anomaly detection through autoencoders using summary statistics-based thresholding. Scientific Reports **14**, 26704 (2024)
10. Li, T., Sahu, A., Zaheer, M., Sanjabi, M., Talwalkar, A., Smith, V.: Federated optimization in heterogeneous networks. Proceedings of Machine Learning and Systems **2**, 429–450 (2020)
11. Li, X., Jiang, M., Zhang, X., Kamp, M., Dou, Q.: FedBN: Federated learning on non-IID features via local batch normalization. In: International Conference on Learning Representations (2021)
12. Lindauer, B.: CMU CERT insider threat dataset. Carnegie Mellon University (2020)
13. Lundberg, S., Lee, S.: A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems **30** (2017)
14. van der Maaten, L., Hinton, G.: Visualizing data using t-SNE. Journal of Machine Learning Research **9**, 2579–2605 (2008)
15. McMahan, B., Moore, E., Ramage, D., Hampson, S., Arcas, B.: Communication-efficient learning of deep networks from decentralized data. In: Proceedings of the Artificial Intelligence and Statistics (AISTATS). pp. 1273–1282 (2017)
16. Mothukuri, V., Parizi, R., Pouriyeh, S., Huang, Y., Dehghantanha, A., Srivastava, G.: A survey on security and privacy of federated learning. Future Generation Computer Systems **115**, 619–640 (2021)
17. Purohit, S., Govindarasu, M.: Federated learning based anomaly detection for distributed energy resource communication. In: IEEE Power and Energy Society General Meeting. pp. 1–5 (2024)
18. Tan, A., Yu, H., Cui, L., Yang, Q.: Towards personalized federated learning. IEEE Transactions on Neural Networks and Learning Systems **34**(12), 9587–9603 (2022)
19. Vu, L., Tran, T., Nguyen, Q.: A new federated learning system with attention-aware aggregation method for intrusion detection systems. In: Advanced Machine Learning for Cyber-Attack Detection in IoT Networks, pp. 227–247 (2025)
20. Wei, K., Li, J., Ding, M., Ma, C., Yang, H., Farokhi, F., et al.: Federated learning with differential privacy: Algorithms and performance analysis. IEEE Transactions on Information Forensics and Security **15**, 3454–3469 (2020)
21. Zhao, Y., Li, M., Lai, L., Suda, N., Civin, D., Chandra, V.: Federated learning with non-IID data. arXiv preprint arXiv:1806.00582 (2018)
22. Zhu, L., Liu, Z., Han, S.: Deep leakage from gradients. In: Advances in Neural Information Processing Systems. vol. 32 (2019)